

Week 4 — From Model to Reusable Component

Turning the Week 3 saved model into a packaged `weather_model` component.

The problem

A saved `.pkl` file works, but it is awkward to reuse.

- Every script must know the file path
- Every script must load it with `joblib`
- Every script must build the input correctly
- Every script must interpret the raw output

What breaks

Change one detail, and every copy of that code breaks.

- Retrain with a new feature
- Rename the model file
- Change the label encoding

The mechanics are spread across the whole codebase.

The goal: a component

A component hides the mechanics behind one method.

- Load the model once
- Call `predict()` with plain dictionaries
- Read back a clear result

Callers depend on the interface, not the internals.

Three lines to use it

```
from weather_model import WeatherClassifier  
  
clf = WeatherClassifier("models/weather_classifier.pkl")  
results = clf.predict([day])  
print(results[0].label, results[0].probability)
```

This is the Week 1 payoff

The same idea as the `RunningConditions` class.

- Configure once
- Apply many times
- One method that callers depend on

Load once in `__init__`

Loading a `.pkl` from disk is slow.

- `__init__` runs once, when the object is created
- It is the right place to load the model
- `predict()` then reuses the loaded model every call

The Prediction result

We define what a prediction is before writing the class.

```
@dataclass
class Prediction:
    label: str # "good" or "skip"
    probability: float # 0 to 1
```


Why a dataclass

`result.label` reads better than `result[0]` .

- Each value has a name
- The class documents what each field means
- These names are used by later weeks

The class skeleton (1 of 2)

```
class WeatherClassifier:
    def __init__(self, model_path: str | Path):
        model_path = Path(model_path)
        if not model_path.exists():
            raise FileNotFoundError(
                f"No model file at {model_path}."
            )
        self.model = joblib.load(model_path)
```

The class skeleton (2 of 2)

```
def predict(self, days: list[dict]) -> list[Prediction]:  
    if not days:  
        return []  
    frame = pd.DataFrame(days)  
    # validate, select features, run model  
    return predictions
```

What `predict()` handles

The method absorbs details the loose script pushed onto callers.

- Empty input returns an empty list
- Missing features raise a clear `ValueError`
- Features are selected in a fixed order
- `1` and `0` become `"good"` and `"skip"`

Fixed feature order matters

A pipeline gives wrong answers if columns arrive out of order.

- Selecting features by name fixes the order
- The caller can pass keys in any order
- A missing feature is caught, not silently misaligned

The four feature keys

Use these exact names.

- `temperature_2m_max`
- `temperature_2m_min`
- `precipitation_sum`
- `wind_speed_10m_max`

Packaging: the layout

```
weather_model/  
├── __init__.py    <- public entry point  
└── classifier.py  <- the class and dataclass
```

The `__init__.py` decides what callers can import.

The public entry point

```
# weather_model/__init__.py
from weather_model.classifier import (
    WeatherClassifier, Prediction, FEATURES
)

__all__ = ["WeatherClassifier", "Prediction", "FEATURES"]
```


The `conftest.py` fix

An empty `conftest.py` at the project root lets pytest import the package.

- Without it: `ModuleNotFoundError: No module named 'weather_model'`
- The same fix used in Week 1
- The file is empty; its presence is what matters

Testing with pytest

A fixture trains a tiny model and saves it to `tmp_path`.

- The tests do not depend on any committed `.pkl`
- Each run starts clean
- The suite is fast and self-contained

What the tests cover

The suite pins down both paths.

- Good day is "good", bad day is "skip"
- Probability is between 0 and 1
- One Prediction per day, in order
- Missing feature and missing file raise clear errors

Prove the tests can fail

Break one thing on purpose, then undo it.

- Return `"skip"` for every day
- Confirm the good-day test turns red
- A test that cannot fail protects nothing

The script

`predict_weather.py` uses the real saved model.

```
def main() -> None:
    clf = WeatherClassifier("models/weather_classifier.pkl")
    for day, result in zip(SAMPLE_DAYS, clf.predict(SAMPLE_DAYS)):
        print(day, result.label, result.probability)

if __name__ == "__main__":
    main()
```

Where this leads

Weeks 10 and 11 import this exact component.

- `from weather_model import WeatherClassifier`
- Load the model once
- Call `predict()` on batches of database records

Because you packaged and tested it, the pipeline can depend on it.